

ro. New or Premium customers can try Duo Enterprise for free by signing up for a GitLab Ultimate trial [here](#). Capabilities included in Duo Pro and Duo Enterprise are listed [here](#).

| Subscription type | Process |

| ----- | ----- |

| I am new to GitLab | Follow the instructions above to create your GitLab.com account and request an Ultimate trial. Then follow these instructions. |

| SaaS Premium | Follow these instructions to request a Duo Pro trial and assign the seats. If you want to trial Duo Enterprise, you can do this by signing up for a GitLab Ultimate trial [here](#).|

| SaaS Ultimate | Follow these instructions to request a Duo Enterprise trial and assign the seats. |

| Self Managed Community Edition / Self Managed Enterprise Edition Free | To trial Duo, you will need to create a GitLab.com account dedicated to the trial (follow the 'I am new to GitLab' instructions). Note: you do not have to add projects in this account to trial Duo Pro features in your IDE. Simply generate a personal access token to configure the supported IDE extension. You will be able to use Code Suggestions and Duo Chat. |

| Self Managed Enterprise Edition Premium | Request a trial using this online form and follow these instructions to trial Duo Pro. If you want to trial Duo Enterprise, you can do this by signing up for a GitLab Ultimate trial [here](#).|

| Self Managed Enterprise Edition Ultimate | Request a trial using this online form and follow these instructions to trial Duo Enterprise. |

Learning how to use the platform

No matter if you are using GitLab for the first time or willing to improve your current usage, these resources will help you get started. Learning best practices will help you gain proficiency during your trial:

- Go through the extensive documentation and review the tutorials
- GitLab University: A collection of trainings and certifications (both free and paid)
- GitLab Youtube channel: Find demos, use cases, recorded webinars and more
- The resource page for reports, ebooks, webcast, etc
- GitLab Blog: search for topics of interest including tips, use cases, etc
- Join the community and become a member through various programs
- Ask your questions and find answers from existing GitLab users on the GitLab Forum
- If you are already a Premium customer, you can get help from the support team

What happens once the trial ends?

If you are trialing on your Premium SaaS Namespace or Premium Self Managed instance, once the free Ultimate trial expires, the account will downgrade back to Premium.

If you are trialing in a new namespace or instance, the account will downgrade back to the Free tier. No data will be lost, but paid features will not be accessible anymore.

If you are satisfied and wish to purchase or upgrade, you can do so on GitLab.com inside your account. For Self Managed, you can purchase the subscription and receive an activation code.

SECTION: solutions-architects/tools-and-resources/workshop.md

Hands-on Workshop

Hands-on GitLab workshops are interactive events which enable GitLab prospects and customers to learn more about GitLab, then apply that knowledge hands-on within a provided GitLab environment. GitLab Solution Architects often conduct the workshops with topics such as DevOps Automation, Portfolio and Project Management, Security and Advanced CI/CD. Workshops are ideally limited to 200 or fewer attendees in order to provide stable working environments as well as to be able to field Q&A from attendees in a timely manner.

In order to carry out a hands-on workshop, a GitLab instance must be stood up for the attendees to carry out the exercises during the workshop. Additional preparation is required to setup the class project and issue tracking workflow.

Creating a Workshop Instance

To create an instance of a workshop navigate to the Workshop Classes Group in GitLab.

1. Click the 'New Project' button
1. Select 'Create from template'
1. Select 'Group' then select the required workshop template
1. Enter a project name that follows the following naming convention:
[Date of workshop]-[Workshop Name]-[Client Name(if applicable)]
i.e. "20210112-Advanced CI CD-GitLab"
1. Click the 'Create project' button

How to Use the Workshop Template

The template consists of several main parts:

The Project Repository

The project Repo contains all the materials required to run a workshop.

The Readme File

The Readme file within the project repository describes the workshop requirements and delivery information.

Tracking Issues

There are 7 issues created to help set up, track and deliver the workshop

Task 1 is the core issue. It is recommended that to use this to consolidate and coordinate with the team. Point team members to this issue to get the information on the particular instance of the workshop. Set timelines and set the dates for all the issues as it relates to the targeted delivery date.

Tracking Milestone

Milestones are used to see progress on workshop set up. They can also be used as part of the group roll up to see across multiple workshop executions.

Tracking Boards

The following boards have been created to help manage the workshop. Feel free to create new boards as needed:

- Tracking - This board uses the flow of scoped labels - Waiting to Start, In Progress, Under Review
- Assistance Needed: This board follows blocking issues affecting workshop setup and delivery

Labels

The following labels are available at the project level - feel free to add new labels as needed, but please do not remove these labels:

- Assistance Needed: a label to flag when help is needed from the team and/or management to resolve an issue
- Content Creation - a label to indicate content is being created or updated
- Waiting to Start - a label indicating no work completion to date
- In Progress
- Under Review

SECTION: solutions-architects/tools-and-resources/pov/_index.md

Proof of Value (POV)

Proof of value is a combination of technical evaluation and the communication of the expected business value of a solution. The solution, its practical application, and how it drives specific business value are proven and documented through the POV process.

Solution Architects are instrumental in guiding prospects and customers to carry out a successful Proof of Value. POV's should focus on specific prospect/customer business outcomes that cannot be achieved through other consultative interactions.

Any GitLab product evaluation should remain separate from GitLab high availability architecture and implementations.

A POV is conducted with a GitLab trial license either using GitLab.com or a self-managed instance. Due to cost, time, and the associated opportunity risk, a POV should not be the default course of action for GitLab prospects/customers.

POV Guidelines and Requirements

- The typical duration for a POV is between 1 and 8 weeks which depends on complexity and style

of engagement.

- A GitLab Customer Success collaboration project developed from the POV plan is the default method for POV management. The project template is only accessible by GitLab team members, but once a collaboration project is created, the customer will be granted access to that project. When utilizing a collaboration project, follow the instructions in README.md closely to configure the entire project properly. The POV template project also contains a wiki page with a POV Plan template that can be used to document the scope, criteria, activities and progress of a POV.
- GitLab Solutions Architects should limit the number of concurrent POV's the SA is engaged in to provide the optimal focus for successful opportunity outcomes. In collaboration with the sales team and their manager, the SA should prioritize all SA activity, including POVs, based on workload, opportunity qualification, and regional or segment strategy.
- A POV must be fully qualified before commencement
- A POV must be tracked in Salesforce
- A POV should not be used to implement a scaled architecture for the customer's unique infrastructure
- A POV is not a full or production quality implementation of the solution
- A POV is not a self run, technical trial to test out a feature for the existing license (can be covered in the CSM cadence for expansion on adoption)
- A POV is not an individual team member on the customer side performing self-learning and certification (can promote PS education services)
- A POV is not a self-paced exploration of GitLab (this can be covered in the CSM cadence)
- A POV is not an informal review of GitLab by a new logo prospect

POV Qualification

- Must tie to an opportunity for a new logo or up-tier to ultimate for an existing customer with a net ARR over 100K or with significant LAM
- It is guided by GitLab SA to run the scoped hands-on technical evaluation with the prospect/customer, collaborate with the sales (SAE, AE) with defined business outcomes and GitLab values
- Prospect/customer has identified and provided available technical and business resources to perform the evaluation, discuss / assess the results to map to the outcomes and value
- Top business drivers are identified with POV scope, success criteria of the POV are defined to execute the POV and subsequently drive next step in the deal process
- Must identify the champion and economic buyers and they must be in agreement (for example, "sign-off") on the success criteria; facilitate the executive connection and sponsorship
- A Command Plan is populated in Salesforce for the opportunity with the following fields:
 - Why Now
 - Why do anything at all
 - Metrics
 - Decision Process - stakeholders
 - Pain
 - Access to economic buyer
 - Impact of not solving that(Compelling event with business needs)

- Top customer business values mapped to one of the GitLab solutions: DevSecOps, Software Compliance, Automated Software Delivery, and cumulatively DevOps Platform.

POV Scope and Acceptance

SA working with SAE and AE can define the POV scope with the customer, with alignment to the business values and the GitLab solution. For each solution, the typical scope and acceptances are listed for reference but the team should define the scope, time and execution with acceptance for each engagement.

- DevSecOps
- Software Compliance
- Automated Software Delivery
- Artificial Intelligence
- and cumulatively, The DevOps Platform

Tracking a POV in Salesforce

Salesforce Object

A POV, no matter the type, should be validated by the Solutions Architect. Therefore, the SA Validated Tech Evaluation Start Date field on the Opportunity needs to be filled with the date the POV would start. Likewise, the SA Validated Tech Evaluation End Date field should be entered when it completes.

In order to track a POV correctly in Salesforce, the Strategic Account Executive should position the opportunity as Stage 3. The Solutions Architect will create the POV object within SFDC when the prospect or customer has indicated interest in moving forward with a POV.

To track a POV, click the Proof of Values tab from the top menu bar in Salesforce. Create a new POV using the New button. Alternately, the Solutions Architect may select the relevant Opportunity, scroll down to the related list labeled Proof of Values and click on the "New Proof of Value" button. This will automatically associate the POV with that Opportunity while all other fields need to be manually completed.

Complete the following fields at minimum:

- POV Owner - this is the Strategic Account Executive
- Customer Success Manager - this is the CSM who will be introduced during the POV
- Solutions Architect - this is the SA who owns execution of the POV
- Proof of Value Name - this commonly is named identical to the opportunity name
- Account - this is the company name
- Opportunity - this is the associated opportunity name (this opportunity should be in stage 3)
- POV Start Date - the date the POV is expected to begin
- POV Close Date - the date the POV is expected to End
- POV Type - this is style POV being executed (deprecated)

Once the POV begins, the Solutions Architect should change the Status field from New to In Progress. When the POV is complete, the Solutions Architect should change the Status to Closed and the Result should be identified as Successful or Unsuccessful. Freeform notes should be

added to support the reason for the successful or unsuccessful result.

Sales and Solutions Architecture Leader Approval Process

When an opportunity is well-qualified for a Proof of Value, Enterprise opportunity win rate is over 90% versus around 30% for other technical evaluations (for example, tech eval, trials, proof of concept).

The field currently records a considerable volume of technical evaluations as Proof of Values. Recording non qualified activity as a POV makes it challenging to track where we can up-level our strategy in an opportunity for a qualified Proof of Value or Value Stream Assessment.

Without the ability to efficiently report on the technical win activity and proof of value work, it's difficult for Sales and SA leadership to identify coaching opportunities and better predict outcomes.

An approval workflow is rolling out to allow Sales and SA leaders to regularly review our activity while marking well-qualified technical win activity as Proof of Values.

Please work closely with your SA or Sales leader counterpart to regularly review opportunities with a Proof of Value object while marking well-qualified POVs with approval.

Sales and SA leader participation in the POV approval process will ensure we can quickly and regularly identify opportunities for coaching and predictability.

Approval Process

- Upon mutual agreement with the associated SAE, the Solution Architect working on the opportunity will create a POV object on the opportunity.
- When a new POV object is created, a notification is posted to the troops-pov-created slack channel.
- In addition to a weekly review of a region's ongoing POVs and technical win work, Area sales and SA leaders monitor the troops-pov-created slack channel for new POVs. When a new POV is created, asynchronous or synchronous collaboration on the quality of the opportunity commences.
- The ASM or SA Manager marks the POV for approval via the Salesforce object or with a troops action in Slack.
- The ASM or SA Manager will query about plans to position professional services at this stage. All SAEs/SAs are required to position Professional services in opportunities with a qualified POV to guarantee continued success for the customer. Please read more about how to position [here](#)

POV Best Practices

Solutions Architects are the owners of the POV, guiding prospects through a successful experience with GitLab. As such, Solutions Architects should be the primary contacts for questions and issues experienced by the prospect during the POV. If support is required for an unforeseen technical issue, some POVs may qualify as priority prospects and may contact the support team.

Many prospects are tempted to include implementation of GitLab high availability as part of a

POV. In this case, the implementation components should comprise a separate POV entirely, separating GitLab functionality evaluations from implementation, load and performance components.

Other best practices for POV's:

- SA's must be part of proposing or committing to POV's
- POV's should be a late-stage component that triggers the sale, not an early offering
- Ask WHY a POV is required vs Q&A or other considerations, since prospects are often using existing GitLab installations
- Feature/function POV's should be run in basic environments, not HA, unless the HA environment is already established and proven
- Required capabilities should be a limited number, 5 or less ideally - these should be purchase triggers and differentiators
- Please work together(SA & SAE) to map required capabilities to decision criteria in the COM plan
- A POV does not implement a fully designed and configured ecosystem in a customer environment, it is a product evaluation

Below is best practice guidance for conducting each type of POV. These processes share a similar goal of meeting identified business outcomes for the customer, but they vary based on engagement style, POV duration, location and intensity. The following applies to all POV's:

POV Key Players

Customer Roles

- Executive contact - A leader with business level, budgetary buy in
- At least one pilot team to execute the POV
- Technical POV lead

GitLab Roles

- Strategic Account Executive (SAE) or Account Executive (AE) - Relationship manager, owner of licensing
- Solutions Architect (SA) - Primary technical contact, POV owner and project manager
- Customer Success Manager (CSM) - Only for qualified accounts. Introduced for customer visibility, occasional POV assistance
- Professional Services - rarely as needed in a pre-sales capacity
- Support Team - only if needed for technical errors, engaged via the customer per the Internal Support page

POV Kickoff Checklist

- SA: Ensure the customer architecture is prepared to support the POV (if self-managed)
- SA: Ensure customer network has access to GitLab.com (if SaaS evaluation)
- SA: Customer Success project is created in GitLab as per the CSM Handbook page
- SA: POV document is created if this is required by the customer, otherwise default to the Customer Success project
- SA: Ensure POV goals and business outcomes are clearly identified prior to kickoff

- SA: For the largest strategic opportunities, notify GitLab Support of POV dates, customer, and other relevant information using the applicable Slack channel related to Self-Managed or GitLab.com support
- SAE/AE: Opportunity updated in Salesforce, set to Stage 3-Technical Evaluation, with POV Information entered per the handbook
- SAE/AE: Signed NDA by the legal team if required
- SAE/AE: Review Customer Assurance Package with customer (if SaaS evaluation)
 - The Customer Package requires an NDA, and contains commonly requested security information (SOC2, Penetration Test Results, etc) and can be requested with Field Security Issue
- SAE/AE: Schedule Internal kick off meeting (detailed below)
- SAE/AE: Schedule kickoff meeting with customer
- CSM: Review collaboration project content prior to internal kickoff meeting
- SA: For SaaS trials that need CI/CD, request trial runner activation per the handbook.

POV Meeting Recordings

POV-related calls may be recorded with customer consent. Recordings may be stored in Chorus, in a folder on Google Drive (if recorded locally), or within the project repository (if small). Any recording links should be identified in the notes stored within the Documents directory of the project repository.

POV Types

There are multiple options when executing a POV. The POV type chosen should reflect the wishes and best fit for the customer. POV types, in order of usage frequency, include:

- Guided - This is the most common POV for Enterprise customers, comprising the majority of all customer experiences, typically lasting between 14 and 45 days, though it may be extended in some circumstances up to 60 days.
- Lite - This type of POV is used exclusively for Commercial opportunities requiring minimal touch points throughout the POV. This is not to be used for Enterprise "guided trials."
- On-site - This less common type is an intense, condensed POV typically executed on the client premises within a tight evaluation window (typically 1 week).
- Paid - Rarely, a prospect pays GitLab for a long duration evaluation of the software (typically exceeding 60 days).

Best practices specific to each type of POV follows.

Guided POV

The Guided POV is the most commonly utilized type of POV for Enterprise accounts. These will commonly have a 30 to 60 day duration.

Guided POV Template Document

For a guided POV, the SA must utilize parts or entire Guided POV document which validates POV dates, success criteria and assumptions of both GitLab and the prospect. The 4 fields below are mandatory for strategic POVs:

- Desired future

- Desired business outcomes
- Required Capabilities (Success Criteria) aligned to decision criteria & value drivers in SFDC
- Metrics

These POV's are marked by regular touch points and consistent interaction over time without requiring full time dedication to the GitLab evaluation on behalf of the customer. It is common to have kickoff meetings, technical support calls, weekly retrospective calls and POV conclusion calls when running a Guided POV. These meetings may be represented by the following suggestions:

Internal Kickoff Meeting led by the Solutions Architect

GitLab Attendees: Strategic Account Executive, Solutions Architect

Agenda:

- Collaboratively review customer success project README to ensure everything is correct
- POV document review (only if a document is required for the POV)
- Discussion of strategy, whether GitLab Support or Professional Services need to be notified
- Strategic Account Executive to schedule external kickoff with customer

External Kickoff Meeting (Remote) led by the Solutions Architect

Attendees:

- GitLab: Strategic Account Executive, Solutions Architect
- Customer: Executive contact, Technical POV lead

Agenda:

- Screen share with the customer to discuss the below agenda items
- Collaboratively review customer success project README to ensure everything is correct
- Agree on due dates for POV completion, add to project POV milestone
- Review project POV issues - add new issues if necessary with agreed due dates
- Review POV document with customer (only when required)
- Establish cadence with customer, Strategic Account Executive to schedule during this call
- Create issues for each cadence call with customer under the POV milestone for call notes
- Provision licenses and establish if customer needs help getting GitLab set up and configured

Weekly Retrospective call led by the Solutions Architect

Attendees:

- GitLab: Solutions Architect, Strategic Account Executive
- Customer: Technical POV Lead

Agenda:

- What went well?
- What problems do we need to overcome?

- What do we need to change?
- Review success criteria - are we on track?

POV Conclusion Meeting led by the SAE or AE

Attendees:

- GitLab: Strategic Account Executive, Solutions Architect, Customer Success Manager
- Customer: Executive contact, Technical POV Lead

Agenda:

- Did we satisfy the required capabilities?
- Did we meet the value driver for the customer? Is this a technical win?
- Identify next steps
- Send out POV survey (where applicable)

On-site or Hands-On POV

The On-site or Hands-On(virtual) POV is typically the shortest and most intense POV. It is critical that before this type of POV begins:

- All outcomes and objectives are agreed to and documented
- GitLab and customer team members are identified
- Calendar schedules are arranged preferably with meetings(like standup) every day
- System installations and/or user administration tasks are complete

The SA will typically join the client at their chosen location(or virtually) and work directly with the team there to quickly identify the value proposition of GitLab within their environment. The SA will commonly dedicate themselves to the client for a fixed few hours every day(or agreed upon cadence) of the week for this POV.

The objective of the daily(regularly cadenced) meetings will be:

- Checking on the POV status
- Identifying blockers & coming up with a plan to remove them swiftly(in an agreed timeline)
- Collaborating on value drivers
- Assisting in solving problems
- Enabling customer POV owners on required knowledge to obtain identified POV outcomes

Paid POV

The Paid POV is less common than other types of POV's. These will commonly have duration greater than 60 days, and the customer will pay for usage of GitLab for the duration of the POV. Before this type of POV can begin, it requires a GitLab prorated licensing purchase to be completed. These POV's are marked by regular touch points and consistent interaction over time without requiring full time dedication to the GitLab evaluation on behalf of the customer. This type of POV will commonly have a larger ecosystem focus, where the value of GitLab is dependent on interactions with other tools and environments within the client's ecosystem. It is common to have kickoff meetings, technical support calls, cadenced retrospective calls and POV

conclusion calls similar to those identified for a Guided POV.

Lite POV - Exclusive to Commercial (DEPRECATED)

When a prospect has an internal POV process to follow, or when time is of the essence, a Lite POV is utilized.

Lite POV Template Document

For a Lite POV, the SA may utilize the Lite POV document (only accessible to GitLab team-members), which validates POV dates, success criteria and assumptions of both GitLab and the prospect.

In the case of a Lite POV, the Solutions Architect is expected to be the sole GitLab contact. "Lite" is determined case-by-case by the size of the prospect as well as their ability to engage with GitLab.

Using the LITE POV Template

To use the Lite POV template, begin by making a copy of the main document for each POV.

Edit each area highlighted in yellow within the document to include pertinent information for any particular prospect. This information includes basic data like the prospect name and GitLab team-member details, as well as data to be collaboratively identified with the prospect, such as primary objective, required capabilities and the environment information. Delete any red-colored instructional text.

Finally, ensure both GitLab and the prospect have a copy of the document. Schedule weekly meetings for the duration of the POV with calendar invites prior to distributing the GitLab Enterprise Edition license for the POV.

Commercial Sales POV Guide

Commercial Sales POV's are commonly executed as a variety of the Lite POV, though they may not utilize the Lite POV document. Typical customer interactions for Commercial POV's are identified below.

Kick Off Meeting

- Duration: 30 Minutes
- Attendees: GitLab Account Executive, GitLab Solutions Architect, Prospective Customer
- Agenda:
 - Define success criteria (as a best practice, have no more than 5 required capabilities)
 - Confirm start and end date
 - Determine primary communication method
 - Customer to decide if they are interested in communicating on a collaborative GitLab project or email only
 - Collaborative, time-limited Slack channels are an option depending on SA capacity and opportunity IACV
 - Frequency options: 30 minute weekly call or email weekly with calls scheduled as needed

- SA role:
 - Acts as the primary point of contact throughout POV process
 - Tracks status updates regarding success criteria
 - Records any existing or new feature requests of interest to customer
 - Leads troubleshooting and escalation of blockers

Commercial Sales - POV and Customer Success Plan Creation

- Create and track the POV in Salesforce according to this process
- Create a new customer project in the Commercial/Pre-sales group. To use the template, select the 'Create from Template' tab, select the 'Group' templates, and then click 'Use template' for the 'New Customer Project - Commercial Sales' template.
- Edit README.md with information specific to POV under the Proof of Value section and fill in any additional relevant fields.
- Upon completion of POV, update Salesforce record with POV result as successful or unsuccessful and provide supportive reasons in the associated freeform fields
 - For a successful POV:
 - SA adds any other known information in README.md, then moves the project from the Pre-sales group to Commercial
 - CSM and AE determine CSM eligibility and engagement based on Account Tier and IACV.
 - CSM, SA and AE schedule meeting with new customer to establish a starting baseline, review the project data and establish intended collaborative usage moving forward.

POV Template Document

As an alternative (or in addition) to using a collaboration GitLab project, a document is available which helps outline the details of a POV. The POV template document (only accessible to GitLab team members) provides the framework for a successful POV by addressing the primary business value driver, the current situation, the desired objective, the required capabilities, metrics and environment information.

This document suggests and verifies specific success criteria for any POV, as well as outlining a mutual commitment between GitLab and the identified prospect parties. It also specifies the limited timeframe in which the POV will occur.

Using the POV Template Document

To use the full POV template, begin by making a copy of the template document (only accessible to GitLab team members) for each POV.

Edit each area highlighted in yellow within the document to include pertinent information for any particular prospect. This information includes basic data like the prospect name and GitLab team-member details, as well as data to be collaboratively identified with the prospect, such as primary objective, required capabilities and the environment information. Delete any red-colored instructional text.

Finally, ensure both GitLab and the prospect have a copy of the document. Schedule weekly meetings for the duration of the POV with calendar invites prior to distributing the GitLab Enterprise Edition license for the POV.

POV Template Wiki Page

As an alternative to using a Google document for a POV plan, a wiki page is available in the collaboration project to capture the details of a POV. This wiki page is similar to the POV template document except in a GitLab Wiki page.

The advantages of using a wiki page are:

- Accessible to customers that may block access to Google docs
- Provides a rich-text document format suitable for sharing with executives after the completion of the POV during the negotiation phase which issues do not
- Provides an ability to have multiple POVs within a single customer project
- Gets the prospective customer thinking about using GitLab for everything

Using the POV Template Wiki Page

For new customers without a collaboration project, the POV Plan template will be included in the Wiki when the collaboration project is created from the POV project template.

For existing customers there are two options for getting the POV template into the existing customer collaboration project:

1. Open the POV collaboration project wiki page and copy the text and paste it into a new page in the customer collaboration project wiki and update it with the specific customer details

1. Clone the POV template wiki locally, update it and push to the customer collaboration project following the steps below (assuming the customer name is MyCustomer):

1. Access the wiki (Plan -> Wiki) page in the POV project template

1. In the upper right corner click Clone repository, copy the clone URL and from a terminal on the local machine run `git clone <cloneurl>`

1. `cd poc-plan.wiki`

1. Rename customer.md to MyCustomer.md (obviously substitute the real customer name for MyCustomer)

`git mv home/customer.md home/MyCustomer.md`

1. Rename the customer directory to MyCustomer

`git mv customer MyCustomer`

1. Rename the pov-template.md file to whatever the pov should be named

`git mv MyCustomer/pov-template.md MyCustomer/devsecops-pov.md`

1. Edit home.md to change link to MyCustomer directory

1. Edit MyCustomer.md to update references

1. Edit devsecops-pov.md to find and replace all references to CustomerName with MyCustomer and any other edits the want to make

1. Stage the changes in git

`git add .`

1. Commit the changes to the local report

`git commit`

1. Point the git origin at the new customer success collaboration project

`git remote set-url origin git@gitlab.com:gitlab-com/account-management/western-north-america/MyCompany/MyCompany.wiki.git`

1. Push the changes to the new customer success collaboration project

`git push origin`

1. View the new wiki in the new customer success collaboration project to ensure it is there and continue making any other edits in the wiki web editor (or on the local machine).

POV Trials

POV usually require a license activation. Instructions for handling trials, extensions and plan changes on [GitLab.com](https://gitlab.com) [here](#).

POV Resources

Below is an evolving list of projects that have proven beneficial during POVs, which may be a great starting point to offer customers.

End-to-End Proof Projects

These projects have a very simple set of code that provides the ability to demonstrate the happy-path for a POV. While these are more in the Hello World category of projects, they tend to have simple automations to exercise different parts of GitLab. SAs have used these in the past as a way to assess the installation of self-managed environments.

- Insecure Tanuki Tech Project was developed internally to show the usage of Auto DevOps. It is predominantly focused on Secure features, but serves well to present all stages.

Demonstrative of Specific Stages

These projects are demonstrative of specific stages. They are generally built on existing code OSS bases which the customer may be familiar with, are easy to understand, and have good literature to refer to.

Secure Stage Projects

- NodeJS Juice Shop repository comes with a `.gitlab-ci.yml` file to get started with SAST and Dependency Scanning. Incorporate others incrementally as needed.
- OWASP WebGoat.NET repository can be enabled with SAST, License Management and Secrets Scanning very quickly using the packaged templates. This validates our positioning in .NET application development (both Framework and Core).

Proof of Value Enablement

- PoV Overview Course
- Executing an Exceptional PoV Course

SECTION: solutions-architects/tools-and-resources/pov/ai.md

Other AI Resources: [Lab](#) | [Demo](#) | [Guided Trial](#) | [POV](#) | [Education Services](#) | [Professional Services](#)

AI POV best practices

We need to proactively identify languages, testing questions and teams with them. Below are the best practices to review before the start of the POV:

- Small team size
 - Languages which has a lot of open source contributions like Java, Python
 - Proactive testing scenario
 - Time boxed hands on POV with active guidance throughout
 - Conduct a pre survey before the POV and post survey after the POV to gauge the success
 - Verify which IDEs and versions will be used for the POV
 - Prior to the POV, host office hours to assist in installing the GitLab Duo plugin in IDEs
 - Create a collaboration project to keep track of issues and resolutions during office hours
 - Provide the team lead and users with the Getting Started with GitLab Duo Enterprise document (internal only - can be distributed to the team lead) for all AI evaluations
 - Provide the GitLab University - Duo Learning Track , and this document with quick start links to various Duo topics
 - Kick-off the POV with a workshop for a headstart. The schedule could be the following, for a 2-hour session
 - 5 min - welcome participants
 - 5 min - introduction
 - 20 min discussion about Gen-AI for development, and pitch of GitLab Duo
 - 1h20 min workshop
 - 10 min conclusion and next steps
 - Make it as easy as possible for participants to experiment with the features, using the following recommendations and adjusting to the context:
 - Onboarding process. Minimize the number of steps needed for a participant to opt into the POV.
 - Example process:
 - Survey to gather participants' name, role, GitLab account
 - Participant is enrolled in the POV instance, and given access to the right projects
 - Participant is invited to POV meetings
 - In the meeting invitation, participant finds instructions to access the POV instance, link to pre-POV survey, link to instructions to go through the workshops asynchronously, etc.
 - Make sure that the POV environment is made available to participants for the kick-off session, even if this session doesn't include a workshop.
- Some participants may want to start experimenting with the solution during the kick-off or right after.
- Give simple and clear instructions to ask for help:
 - Where to create an issue and who to tag
 - Invitation to Office Hours
 - Bias for guided, hands-on activities. See POV activities for inspiration
 - When POV goals are to surface quantitative indicators of improvements brought by AI-powered features, it is tempting to adopt a comparative study approach for the workshops or hackathons described above.
 - It consists of assigning the same task to two groups of developers. One group works as usual, while the other is allowed to use AI-powered features. Optionally, a second assignment is given, and the two groups are switched around.
 - A drawback of this approach is that working without AI-powered features isn't very fun. Considering how precious it is in enterprise settings to have time dedicated to this kind of initiative, spending it on working as usual may not be ideal.
 - As an alternative, ask participants to log, for each task
 - Before: a time estimate if they were working as usual
 - After: the time it actually took, with the help of AI-powered features.
 - After: include qualitative feedback about the comfort of working, quality of software, etc.

- Familiarize yourself with Duo Chat best practices and Duo Code Suggestions top tips. There are some great tips and tricks in this blog to assist with integrating Duo Chat and Code Suggestions into a customer's workflow. Check the documentation for more practical GitLab Duo uses case.

Pre-requisites

We need to make sure the customer has gone through the AI hands-on workshop to have a great experience with AI POV. We also recommend building a Customer Success Plan and getting agreement from your customer before the POV start.

Input to the POV

- Example Customer template1
- Example Customer template2
- Streamline your POV by adopting 3 Goals in 30 Days

Setting up the POV instance

Set-up the POV instance with your customer's POV lead or technical counterpart.

- Getting GitLab Duo trial Licenses
 - For existing GitLab customers, follow the steps here to request Duo trial
 - For prospects, request trial licenses here
- Setup the participant environment, their IDE and do a trial run. Below are a few links and documents to help.
 - GitLab University - Duo Learning Track
 - Getting Started Documents - Getting Started with GitLab Duo Enterprise
 - Quick Start - Duo links
- Import the public AI POV Plan project template at the root of the POV group, namespace, or instance.
 - It is intended to be used as a central point of entry for participants to make all useful information and resources easily discoverable for testers.
 - Follow the checklist provided in the REAMDE.md to customize this project
- Configurations needed in the instance or group are better done by the customer's POV lead
 - This way they have ownership and understanding of the configuration
 - It is also on them to review the terms of experimental features to be turned on, and GitLab's testing agreement
- Test AI-powered features in various settings, before undertaking any POV activity. Troubleshoot or adjust POV activities accordingly.
 - within the customer's network or not
 - with a VPN enabled or not
 - with different IDEs including the Web IDE

POV Activities

The following are different workshop formats to pick and combine in order to facilitate and lead the POV.

Examples of POV activity timelines:

- Customer A
 - Week 1: Kick-off and standard guided workshop
 - Week 2: Custom workshop on IDE integration setups and IDE features
 - Week 4: Standard guided workshop for a second set of users
- Customer B
 - Week 1: Kick-off and standard guided workshop
 - Week 3: Custom workshop on improving quality and generating documentation
 - Week 4: Coaching the POV leads to prepare an internal demo
 - Week 6: AI Hackathon, in three separate sessions, with a measurement of productivity improvement (see Comparative Study)

Standard AI Workshop

The standard AI workshop leverages the Tanuki Racing project from GitLab Learn Labs.

- Under Courses / Workshops / AI are the workshop instructions

Depending on the availability of the SA in charge, this workshop is either delivered

- In a synchronous, instructor-lead session, for optimal engagement
- In a self-paced, asynchronous setting, for low-touch engagements, or for participants unable to attend the sync session
 - Instructions in the workshop project are intended to suit this case.
 - A recording of the session may also be made available.

Custom AI Workshop

This approach consists of customizing the standard AI workshop to a customer's specific needs or context.

- Consider exploring specific workflows that matter to the customer.
- Make sure to thoroughly test the workflow before the workshop, and adjust prompts and instructions as needed
- Consider using a different project to support the workshop. Either an open source project, or a customer project could be used to focus on a specific framework or language
 - If a customer project is used, first validate whether or not it is allowed to be uploaded to the POV instance or namespace.

It requires more involvement, both from GitLab's side and from the customer side. Validate this with the POV lead before starting the initiative.

It is also likely to yield higher engagement and enthusiasm from the participants. And to give decision makers a more accurate understanding of the value they can expect, including quantitative indicators.

AI-powered Hackathon

This approach goes one step further in terms of involvement, both for preparation, and for the workshops themselves. Workshops are usually 1.5-2.5 hours long, whereas hackathons typically last half a day to a full day.

Participants form teams, and use AI-powered features at will for a given period of time to deliver a prototype.

A hackathon project is prepared to give participants

- Instructions and links to useful resources
- A project boilerplate, helper functions, evaluation functions. Consider having multiple versions for multiple use cases or languages.
- A GitLab issue, issue template, to log time estimates and results. The project's readme may also be used for this purpose

The scope could be

- The same for all teams. Eg "Flight tracking app" or "Competitive quiz app"
- To be chosen within a set of assignments.
- Free: develop whatever you want in the allocated time. Give some example use cases.
- Consider public coding challenges, which are good candidates for hackathon assignments, as they usually give
 - instructions and boilerplates for teams to use.
 - clear success/failure or performance for each tasks, when there is a possibility to submit a solution and get instantaneous feedback.
 - time estimates or public leaderboards for each tasks, which can be useful to estimate the value of AI-powered features
- A caveat to public coding challenges is that they could sometimes be considered too far from the reality of enterprise work. To be discussed ahead of time with POV leads.

Additional guidance:

- Be mindful that a hackathon setting could be intimidating or stressful to participants because of:
 - Time pressure
 - Working on unfamiliar tasks
 - Working outside of their usual tooling
 - Starting from scratch, which may not be frequent in enterprise settings
 - Feeling exposed to coworkers or managers judgment
 - Discovering new AI-powered features
- To mitigate this discomfort:
 - Set expectations: participants are not expected to complete all tasks, or deliver perfect software in such a short timeframe
 - Set rules of conduct for participants, about openness, kindness, communication
 - Prepare resources and tools in the hackathon project
- To make the most out of the allocated time for the hackathon, consider forming teams before the workshop.

Comparative study

When POV goals are to surface quantitative indicators of improvements brought by AI-powered features, it is tempting to adopt a comparative study approach for the workshops or hackathons described above.

It consists of assigning the same task to two groups of developers. One group works "as usual",

while the other is allowed to use AI-powered features. Optionnally, a second assignment is given, and the two groups are switched around.

- A drawback of this approach is that working without AI-powered features isn't very fun. Considering how precious it is in enterprise settings to have time dedicated to this kind of initiatives, spending it on "working as usual" may not be ideal.
- As an alternative, ask participants to log, for each task
 - Before: a time estimate if they were working as usual
 - After: the time it actually took, with the help of AI-powered features.
 - After: include qualitative feedback about comfort of working, quality of software, etc.

Guided Workshop best practices

Before the workshop

- See setup of the POV instance
- Have a dry run, to surface and tackle any issues with the GitLab instance, or videoconference: screen share, breakout rooms, etc.

During the workshop

- Lead the workshop with a co-host from GitLab, who is familiar with the workshop.
 - The co-host can answer questions in the chat, and help some of the students troubleshoot without slowing everyone down.
 - Useful resources include our docs' troubleshooting section, Field FAQ, AI SME FAQ
- Use video conference chat messages to gather feedback with emojis, in order to engage the audience and follow students' progression along workshop steps
 - Ex: "What's your favorite IDE?" "What's the language or framework you use the most", "Workshop project created?", "Pipeline triggered?", "Vulnerability fixed?", etc.
- Keep some time at the end to gather feedback, and prepare next steps.
 - Questions like "What features do you wish to test further?" will hopefully keep students involved after the workshop.

After the workshop

- Ask the customer POV lead about the feedback they had internally
- Give feedback about the workshop, in the appropriate GitLab project or Slack channel

Results

Tracking

Other POV Scope and Acceptance

SA working with SAE and AE can define the POV scope with the customer, with alignment to the business values and the GitLab solution. For each solution, the typical scope and acceptances are listed for reference but the team should define the scope, time and execution with acceptance for each engagement.

- DevSecOps

- Automated Software Delivery
- DevOps Platform cumulatively

AI engagements in general

Discovery - adapt to the customer's context

As early as possible, actively discover your customer's specific context.

For instance, before even presenting GitLab's vision or positioning, ask the customer if they have an existing company strategy or guidelines, or personal opinions on Gen-AI in general, and for development.

Inquire about the following topics

- Vision on Gen-AI
 - Sentiment: afraid of AI vs excited about the technology vs cautious about security implications
 - How much is expected: a complete overhaul of practices, or punctual help for developers
 - Understanding of the market: awareness of the multiple vendors and products? Do they see AI models as a "winner-takes-all" or as a commodity?
- Company strategy
 - Is Gen-AI for software development an isolated effort, or part of a bigger initiative? In the latter case, it might be useful to understand the topics, metrics, timeline, and decision-makers involved.
 - Are other Gen-AI tools already used?
 - Are developers authorized to use Gen-AI tools that were not vetted and purchased by the company? What are the security and privacy implications? Are there control mechanisms in place to prevent this kind of shadow IT?
- Use cases and workflows
 - Providing comfort and productivity for developers (code generation performance)? Or broader vision over the broader picture = SDLC performance.
 - Metrics
 - Objectives and key results
- Constraints. Depending on the answers to the following questions, some AI features might not be immediately available. Or they may not apply to a subset of projects.
 - Self-managed or Dedicated? Cloud licensing possible? Premium or Ultimate?
 - Security and Privacy: allow for API calls outside their infrastructure? Including sending proprietary code?

Examples of customer inputs that had a significant influence on the technical evaluation:

- "Code suggestions are not useful for us, we don't want to activate that feature. Instead, we're looking to leverage GitLab Duo to make the most out of our existing applications: documenting, refactoring, adding tests, innersourcing, etc."
- "We only want to use AI features if they're connected to self-hosted, custom AI models, without any data leaving our network"

Elements of perspective and communication on Gen-AI features

The following elements proved effective to influence positively the way AI-powered features are

perceived.

Pioneer/learning mindset: Gen-AI is relatively new for everyone everywhere. As early adopters, we'll learn the best patterns to interact with it, and be creative to surface new use cases where AI will prove most useful

- Customers might ask for features that are not currently available, and which could sound too ambitious or not technically feasible. Try to be open, don't dismiss these ideas too quickly. Consider how a combination of GitLab's existing or roadmap features could serve this purpose, specifically GitLab Duo Chat which is very versatile.
- Encourage this kind of reflection with questions like: Without considering any technical constraints, what use cases would you like AI to perform? Can you think of any creative use that could be made of the current feature set?
 - This is meant to put attendees in a position to find potential solutions, rather than finding potential issues.
- Examples of creative use cases that can be achieved
 - "I'd like the AI to document legacy applications" could be achieved by customizing the /explain action
 - "I'd like to convert Python code to Java" or "Convert a Jenkins pipeline to GitLab CI". This could be achieved by customizing the /refactor action
 - "I'd like an answer in a language other than English". This typically works pretty well, although our Product team does not actively optimize for multi-language support.

Playfulness: The element of "randomness" in AI can be fun. Early-stage, experimental

- When demonstrating GitLab Duo Chat, encourage attendees to try it themselves, or give you creative prompts.
- Even out-of-context questions like "give me the recipe for apple pie" add some fun, and demonstrate that there are some guardrails in place - AI typically responds that this is not the kind of question they can answer.

AI as an assistant to developers, as opposed to an autonomous software delivery system highlights the importance of the developer's role and skill: review, adjust, use AI as a tool

- Mention the reason behind the name "GitLab Duo":
 - > The name GitLab Duo is rooted in You + GitLab AI = the AI dynamic duo.
- Be mindful that some companies or individuals might have a pre-existing bias against Gen-AI. Some might even see any Gen-AI effort as dangerous, or as a way to replace human workers. If this is the case
 - try to understand why
 - consider giving a presentation to popularize Gen-AI to explain how it works, and explore its strengths and limitations. This could go a long way towards putting it under a more favorable light.
 - Convey the message that any Gen-AI suggestion is only useful if used in the right context, reviewed, and adjusted as needed by an expert.
 - > GitLab Duo is a customer-centric approach focused on privacy first, where customers know their intellectual property is secured.

Expect the unexpected

Gen-AI won't necessarily give the same answer to the same prompt.

You will run into use cases where AI gives unexpected, wrong, or no answers in your demos. Also keep this in mind when leading a workshop, as a portion of the students will probably experience this.

While this could typically be interpreted as "failing" in a regular context, it doesn't necessarily have to be the case for AI conversations. Use the following to mitigate the negative impact this might have, and even turn it to your advantage:

- Manage expectations and mindset, both for a person running a demo or for someone following a workshop
 - Mention that this is expected, and inherent to Gen-AI. It is neither a bug that will be fixed nor something specific to GitLab's Gen-AI features.
- As a presenter, be prepared for these "happy incidents"
 - Be prepared to brush it off or laugh it off. One wrong suggestion is not a big deal. After all, it's not a true demo until something goes wrong.
 - Use them as opportunities to adopt pioneer/learning/playful mindset, or convey the messaging that AI is meant as an assistant to development teams.
 - > "If you want to dig into this later, I'm sure we'll be able to find the right prompt to make this work"
 - > "Good thing I'm following best practices and reviewing AI suggestions"
 - > "Glad to see I'm not obsolete just yet!"
- The unexpected answer might be at least partially useful, or serve another purpose
- Re-try with the same prompt, or with a very slight change
- Have alternative prompts or use cases ready
- If you are presenting in a high-stakes setting where "nothing should go wrong" use video recordings as a backup

SECTION: solutions-architects/tools-and-resources/pov/automation.md

Other Automated Software Delivery Resources: Lab | Demo | Guided Trial | POV | Education Services | Professional Services

The Automated Software Delivery solution is identified for organizations who are trying to reduce the complexity and increase the velocity of Continuous Integration (CI) and Continuous Delivery/Deployment (CD) capabilities to deliver better software faster.

From the early discovery when qualifying the POV, it is confirmed that the current CI/CD pipeline solution does not provide sufficient flexibility in organizing pipelines and requires too much administrative overhead to operate and integrate the CI/CD system with the source code repository.

The key objective of the POV is to validate the value of a tightly connected CI/CD pipeline system that does not require additional integration or plugins to provide a broad variety of pipeline shapes and capabilities.

Input to the POV

- Pain points identified: Toolchain review and mapped out existing app sec tools and usage

- High level ROI discussion identifying potential tool consolidation (hard cost); and overall efficiency gains with transformative process with shift left security
- Both DevOps + App sec team are stakeholders to be involved

Suggested Success Criteria

- Business Driver: increase velocity, consolidate/reduce spending, improve security posture, protect brand and reputation
- Enterprise Initiative and Sponsor: cloud transformation, CIO
- Required capabilities with the objectives to infuse security earlier in the development process, ability to scan all code and act them real time. Have the security oversight as the integral part of the end to end DevOps.

| Required Capability | Acceptance | Objective | GitLab Feature |

| --- | --- | --- | --- |

| Low friction developer experience with CI pipelines | Developers are able to easily run pipelines. Developers are able to modify pipelines using normal development methnods | Continuous Integration | Pipeline per branch, pipeline configuration as code in repo with application, Pipeline structure is intuitive |

| Developers quickly fix problems with immediate and precise pipeline feedback | Pipelines fire on push, on merge, from UI and API | CI, shift left | Pipeline results and code quality report in merge request widget |

| CI support for monorepos | CI only for parts of repo that changed | Ease of dependency management | Parent - Child pipelines, when: changed: directive, DAG pipelines |

| Pipelines scale to match performance and cost demands | Able to run hundreds of pipelines concurrently | Scalability | scalable job queues, scalable runner architecture |

| Coordinated pipeline execution across multiple repositories | A pipeline running for one repo is able to trigger pipelines in other repos and record outcome | Pipeline coordination | Cross-project pipelines |

| Shared and reusable pipelines | Pipelines can be composed from reusable templates and included pipelines | Innersourcing and best practices | Pipeline templates, includes, compliance pipelines |

| Enforce compliance with required pipelines | Mandatory parts of a pipeline can be set up for an entire group of repos | Organizational compliance | Compliance pipelines, Security Scan Policies |

| Control who can run pipelines and under what conditions | Running pipelines against "protect" branches is limited by role | Continuous Deployment | Credentials Inventory, protected branches, runners, variables and tags |

| Limit access to deployment pipelines | Only designated maintainer users can deploy to higher level environments | Continuous Deployment | Protected runners, variables, environments and branches |

Other POV Scope and Acceptance

SA working with SAE and AE can define the POV scope with the customer, with alignment to the business values and the GitLab solution. For each solution, the typical scope and acceptances are listed for reference but the team should define the scope, time and execution with acceptance for each engagement.

- DevSecOps

- Software Compliance
- DevOps Platform

SECTION: solutions-architects/tools-and-resources/pov/compliance.md

Other Compliance Resources: Lab | Demo | Guided Trial | POV | Education Services | Professional Services

When the SA has confirmed with the prospect or the existing customer especially businesses in the regulated industry for a Software Compliance solution, the scope of the POV will focus on securing the software supply chain and simplifying the compliance with common industry regulations while at the same time speeding the software velocity.

The key capabilities to validate to the value are

- Compliance Framework
- Compliance Workflow Automation
- Compliance Dashboard and Audit Event
- Software Supply Chain Security

Input to the POV

- There is a clear requirement on compliance and audit either driven by the regulatory requirements or enterprise standardization

Suggested Success Criteria

- Business Driver: improved compliance and easy audit to be able to stay in market or enter into new market that is regulated by the government or regulatory bodies.
- Enterprise Initiative and Sponsor: new product launch or regulator audit
- Required capabilities with the objectives to infuse built-in compliance into the development process, ability to generate and audit artifacts and evidence for all steps and process.

| Required Capability | Acceptance | Objective | GitLab Feature |

| --- | | --- | | --- | | --- | |

| Compliance policies and framework | Compliance can setup policies and enforce across line of business or across the entire enterprise | Prove to regulators the compliant process | Compliance Framework; Required Pipeline; Compliance Dashboard; Audit Events; Separation of Duties |

| Standardized Software Supply Chain | Enforced standards to support compliance policies in the software supply chain | Organizational management for compliance | Project to group level compliance view |

| Audit | Audit on organization changes and user access | Proactive audit for internal and external audit | Audit event, API, dashboard |

It may overlap with some of the key acceptance in DevSecOps, but more focused on standard compliance and auditability.

If this is the key scope for the POV, reach out to the strategic field team to review the top drivers and regulatory requirements or deadline, and define the right scope and execution steps for the POV with the acceptance.

Other POV Scope and Acceptance

SA working with SAE and AE can define the POV scope with the customer, with alignment to the business values and the GitLab solution. For each solution, the typical scope and acceptances are listed for reference but the team should define the scope, time and execution with acceptance for each engagement.

- DevSecOps
- Automated Software Delivery
- DevOps Platform cumulatively

SECTION: solutions-architects/tools-and-resources/pov/devsecops.md

Other DevSecOps Resources: Lab | Demo | Guided Trial | POV | Education Services | Professional Services

The DevSecOps solution is identified for organizations who are trying to "shift left" to find security vulnerabilities earlier within their DevOps methodology but have not been able to achieve the expected results.

From the early discovery when qualifying the POV it is confirmed that the existing application security is separated from the DevOps flow, causing bottlenecks as the final hurdle in the development life cycle and typically controlled only by the App Sec team.

The key objective of the POV is to validate the value of the shift left security to the developer's hand along with app sec team collaboration at time of app development.

Input to the POV

- Pain points identified: Toolchain review and mapped out existing app sec tools and usage. Here is the example of the discovery for the toolchain and the mapping of the toolchain.
- High level ROI discussion identifying potential tool consolidation (hard cost); and overall efficiency gains with transformative process with shift left security
- Both DevOps + App sec team are stakeholders to be involved

Suggested Success Criteria

- Business Driver: increase velocity, consolidate/reduce spending, improve security posture, protect brand and reputation
- Enterprise Initiative and Sponsor: cloud transformation, CIO
- Required capabilities with the objectives to infuse security earlier in the development process, ability to scan all code and act them real time. Have the security oversight as the integral part of the end to end DevOps.

Required Capability	Acceptance	Objective	GitLab Feature
Approval Process and workflow	Approval rules with security team; Dev + Security Team collaboration	Approval Rules	Vulnerability mgmt
Developer App Sec Workflow Process	Scan all code branches within desired timeframe; Shift left security with productivity improvement	Vulnerability mgmt	Security Test & MR Report
Security team Workflow Process	Security team to drive policies and facilitate triage	Manage app and organizational security posture	Security Dashboard; Vulnerability mgmt; Security trends
SAST security test review	Scan for all detected languages	Shift left developer's experience with real time feedback	SAST analyzers; MR SAST widget; Vulnerability Dashboard
Dependency Scanning	Scan for all detected languages	Shift left developer's experience with real time feedback	SCA analyzers; MR SCA widget; Vulnerability Dashboard; SBOM and Dependency Tree
Container Scanning for Cloud Native	Scan and block known vulnerable images	Recommendation and auto-remediation	Container analyzers; MR Container widget; Vulnerability Dashboard; Auto-remediation via MR
API Security Test	GitLab to review API fuzzing project	Shift left pen test like capability to dev + QA	
DAST API; API Fuzzing			

This tends to be most common POVs for new logos and uptier to ultimate. Ensure you engage the application security team at the customer.

Proof Points Consideration

It is not recommended to carry out application security tests on all types, particularly fuzzing due to the intensive work and expertise required.

Proof points can be obtained to supplement the POV as acceptable results.

It is recommended to provide proof points using GitLab apps and projects such as a sample java app for coverage fuzzing. The proof points can be documented or reviewed with the review session, and considered as part of the acceptance.

If the DevSecOps acceptance maps to the regulatory requirements, it tends to require integration with other security control tools and analyzers, it is important to include that in the POV scope and acceptance but more as discussion with supplemental proof points instead of actual end to end integration.

Other POV Scope and Acceptance

SA working with SAE and AE can define the POV scope with the customer, with alignment to the business values and the GitLab solution. For each solution, the typical scope and acceptances are listed for reference but the team should define the scope, time and execution with acceptance for each engagement.

- Software Compliance
- Automated Software Delivery
- DevOps Platform cumulatively

SECTION: solutions-architects/tools-and-resources/pov/platform.md

Other DevOps Platform Resources: [Lab](#) | [Demo](#) | [Guided Trial](#) | [POV](#) | [Education Services](#) | [Professional Services](#)

The platform value play will provide an overview of the DevOps Platform solution and the pain points it addresses across the enterprise instead of individual stages or silos, outline key questions you can ask to assess the customer's needs and identify the relevant aspects of the solution, and offer paths to other discussions of GitLab's value.

I